Authoritative STM32 Tic-Tac-Toe Server with a hin PC GUI Client

Mosa Mdladla (MNYMOS010) Charmain Madzibandila (MDZMUT005)
EEE3095S CS Project 2026, Department of Electrical Engineering
University of Cape Town

***Abstract*—We chose to build a Tic-Tac-Toe game system in which the STM32 development board acts as an authoritative server and stays in communication with a thin PC-side GUI client all over a UART serial connection. The server enforces the rules through validating and handling commands, move requests, turn order and other aspects of the game before making changes to the board. The communicatino protocol uses message frames that are formated with sequence numbers, error responses and discarding mechanisms for bad input as well as CRC-16 error detection which allows for more reliable communication. The system was tested using automated robustness and hardware tests that reinforce reliable communication throughout the game play. A total of 5/5 of the robustness and 25/25 protocol tests passed, 19/19 of the game-rule validation tests and 6/6 of the hardware tests passed which validates the effective serial communication and protocol error handling.**

I. Introduction

In a small client-server game system, the STM32 must act as the referee and single authority because the PC client can not be reliably trusted to validate moves or display the correct game state at a given point in time. This is due to the fact that it runs the game on a separate system to the STM32 and it could contain software errors that lead to further inconsistencies. As a result, the STM32 microcontroller must be the authoritative server that validates each request the PC client makes and maintains the correct game state independently of where the software is run

Tic-Tac-Toe was chosen as the game system because it is straightforward and has a small board which allows move validation, game state and turn management and game results to be implemented and tested clearly. For the GUI, the PySide6 library was chosen because it provides user-friendly and interactive layouts as well as mechanisms that allow for request/response detection between the client and server. This reinforces and accomodates the need for a separated serial communication that will avoid blocking.

The firmware separatess the protocol parsing, game logic, state transitions, and UART handling into distinct modules. The communication uses frame messages containing a sequence identifier, command, arguments and checksum, which allow requests to be identified and integrity checked. Prior to integration, automated tests verify protocol handling and game rules enforced; when systems have been integrated, the complete communication channel is verified on the physical board.

TABLE I
Task allocation between project members.

Area	Student 1	Student 2
Protocol	Joint specification and agreement	
STM32 server	UART and framing	Game logic and hardware I/O
PC GUI	Serial thread	Board display and handling
Tests	Robustness	Game rules/results
Diagrams	Circuit and GUI	FSM and Sequence Diagram
Report	Joint collaboration and reporting	

Structure. Section II describes the message protocol and communication specifications.. . . Section III describes the system design and state machine. . . Section IV discusses the firmware and GUI implementation.. . . Section V describes server's game-rule enforcement and maintaining state. . . Section VI protocol and robustness tests and error handling. . . Section VII discusses testing and validation testing. . . Section VIII.

II. Requirements and Message Protocol

A. Serial Link and Message Framing

Physical link: which USART, which pins, baud rate, data bits, parity, stop bits, flow control. Framing: the terminator, how you treat `\r`, the maximum frame length and what happens when it is exceeded, the timeout for an incomplete frame, the character set allowed, the checksum or CRC (if any) and how it is computed. Use Table II.

The PC client communicates with the STM32 server through USART1 over the ST-LINK virtual COM port (COM5). The link uses a baud of 115200, no parity and one stop bit (8N1), with no hardware flow control. Each frame follows the following format:

`@AAAA|COMMAND|ARGUMENTS|CCCC\n`

where '@' indicates the start of the frame, 'AAAA' is a four digit hexadecimal sequence identifier, 'COMMAND' indicates the game commands and 'CCCC' is a 4 digit hexadecimal CRC (cyclic redundancy check). The \n is the terminator of the frame and any empty fields are indicated by a '-'.

The CRC is calculated over `SSSS|COMMAND|ARGUMENTS`. The implementation uses CRC-16/CCITT-FALSE with polynomial `0x1021`, initial value `0xFFFF`, no input or output reflection, and a final XOR value of `0x0000`.

TABLE II
SERIAL FRAMING RULES.

Rule	Server behaviour
Physical link	USART1 via ST-LINK virtual COM port; 115200 baud, 8N1.
Terminator	`\n` ends a frame.
Maximum frame length	64-byte parser capacity; overflowing frames are rejected.
Incomplete frame	Timeout value of 500 ms ; no reply is sent for a frame discarded by timeout.
Invalid framing	A framing or parser error produces `NACK|BADFRAME`.
Invalid checksum	A checksum mismatch produces `NACK|BADCRC`.

TABLE III
SUPPORTED REQUESTS AND SUCCESSFUL RESPONSES.

Request	Accepted in	Response
`CONNECT|-`	Any game state	`OK|-`
`NEW|-`	Any game state	`OK|-`
`STATE|-`	Any game state	`BOARD|<board>`
`MOVE|X,i` or `MOVE|O,i`	Game in progress	`BOARD|<board>`, `WIN|<player>,<board>` or `DRAW|<board>`
`RESIGN|-`	Game in progress	`OK|GAMEOVER`
`DISCONNECT|-`	No active game	`OK|DISCONNECT`

Malformed frames and requests with invalid sequence identifiers receive an error response using `0000`. The receive interrupt places incoming bytes into a ring buffer. The main loop passes these bytes to the frame parser which processes complete frames and detects framing errors. The parser has a maximum capacity of 64 bytes. A timeout of 500ms is used to discard frames that are incomplete.

B. Board Encoding

The board is represented by nine ASCII characters, row by row, with "." used for empty cells and X or O for occupied cells. The board is transmitted as a single nine-character argument. For example, the board string `X.O..OXX.` represents the following:

X	.	O
.	.	O
X	X	.

C. Command Set

Every sent request uses the same frame format and structure, with the response type in the command field.

Connection and game setup. The `CONNECT` command acknowledges communication with the server; it does not create a game. The `NEW` command clears the board, sets the current player to `X`, and starts a new game. Both return `OK|-`.

Moves and results. The `MOVE` command specifies the player and target cell, using the format `X,i` or `O,i`, where i is between 0 and 8. A legal non-terminal move returns `BOARD` with the updated board. A winning move returns `WIN` with the winning player and final board, while a full board without a winner returns `DRAW` with the final board. The server determines these outcomes and does not rely on the client to calculate them.

State recovery and termination. The `STATE` command lets a client retrieve the current board after reconnecting or when its local display may be out of date. The `RESIGN` command ends an active game but leaves the board contents intact. The `DISCONNECT` command acknowledges disconnection only when no game is in progress; it is rejected during an active game.

TABLE IV
ERROR RESPONSES AND THEIR CAUSES.

Error	Cause
`BADFRAME`	Invalid frame structure or a parser framing error.
`BADSEQ`	The sequence identifier is not four hexadecimal characters.
`BADCRC`	The CRC field is invalid or the calculated checksum differs from the received value.
`BADCMD`	The command is unknown or its arguments do not match a supported command format.
`NOGAME`	A move or resignation is requested when no game is in progress.
`BADARG`	The `MOVE` argument does not contain a valid player and single-digit cell index in the required format.
`RANGE`	The requested move index is outside the range 0–8.
`NOTURN`	The requested player differs from the server's current player.
`OCCUPIED`	The target cell already contains a mark.
`GAMEACTIVE`	A disconnect is requested while a game is in progress.

D. Error responses and Game State

Errors use the response format `@AAAA|NACK|REASON|CCCC\n`. Table IV shows the implemented error reasons. For rejected game requests, validation occurs before a move is written to the board. As a result, a rejected move does not change the board or the game state.

For a `MOVE` request, the server checks whether a game is in progress first, followed by argument format, index range, player turn and cell occupancy. The first failed check determines the response. For example, a move with an invalid index is rejected with `RANGE` before the turn or occupancy checks are performed.

A sequence identifier is echoed in normal responses and most error responses. If the identifier itself is invalid, the server responds using `0000`. Errors detected by the frame parser also use `0000`. An incomplete frame discarded by the timeout does not produce a response.

E. Departures from the reference protocol

The system's implementation uses a pipe-separated frame consisting of a sequence identifier and CRC, which allows the client to receive responses corresponding to specific requests and the CRC is able to detect corrupt messages using the CRC-16 checksum with four-digit hexadecimal representation. In addition, the server uses specific error messsages that correspond to particular requests such as framing or argument errors. This is all as opposed to the reference's simple message

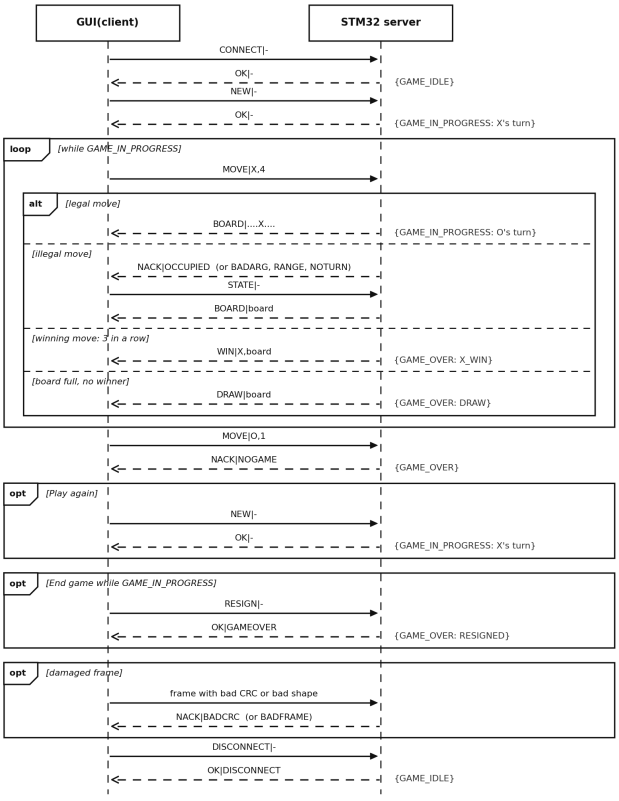


Fig. 1. Message sequence between the GUI client and the STM32 server.

format for serial communication.

For MOVE responses, the implementation returns `BOARD`, `WIN` or `DRAW`, depending on the game results. This helps to decipher a move mid-game and an actual game result.

The `DISCONNECT` command is rejected when `GAMEACTIVE` is true whilst a game is in progress. This ensure thats a game must end first before disconnecting the server.

The `NOTURN` response is tested when a client sends a `MOVE` request, with a player that is requested to be next. The server then compares the requested player and the current player before it attempts to modify the board.

III. Specification and Design

A. STM32 as the authority: Validating moves and Maintaining Game State

The overall systems uses a a client-server architecture in which the STM32 microcontroller acts as the game server and the PC GUI acts as the client. Communication is over a defined message protocol.

The firmware separates serial communication, game logic and hardware interaction into modules. The communication modules receives and sends responses, the game logic enforces the game rules and determiens the outcome of each move and the hardware interaction is related to handling the physical buttons, LED's and LCD. These modules keep low-level hardware operations separate from rule-enforcement.

The PC application is responsible for the user interface and serial communication with the server. It displays the boar, provides controls for connecting to the device and starts/ends a game, sending requests when a user performs an action. As a result, the interface is focused on interaction and user-friendly design whilst the STM32 remains the authority for game decisions.

B. Server Move-Validation

The server FSM (Fig. 2) has three states. `GAME_IDLE` is the power-up state with an empty board. `GAME_IN_PROGRESS` alternates between X's turn and O's turn, driving PB0–PB3 for X and PB4–PB7 for O. `GAME_OVER` holds the result (`X_WIN`, `O_WIN`, `DRAW` or `RESIGNED`), and each result has its own LED pattern.

In `GAME_IDLE`, `CONNECT` replies OK and keeps the board empty, `NEW` clears the board and moves to X's turn, and `DISCONNECT` replies OK. `STATE`, `MOVE` and `RESIGN` are rejected with `NACK|NOGAME`, because there is no game to act on. In `GAME_IN_PROGRESS`, `STATE` returns the board and `MOVE` goes through the validation described below. `RESIGN` replies `OK|GAMEOVER`, moves to `RESIGNED` and leaves the board as it was. `NEW` and `DISCONNECT` are rejected with `NACK|GAMEACTIVE`, so a running game can never be abandoned silently. `CONNECT` is the exception: it always resets the server to `GAME_IDLE`, even mid-game. In `GAME_OVER`, `STATE` still returns the final board, `NEW` starts a fresh game, and `DISCONNECT` and `CONNECT` return the server to `GAME_IDLE`. `MOVE` and `RESIGN` are rejected with `NACK|NOGAME`.

A `MOVE` received in `GAME_IN_PROGRESS` passes four checks in a fixed order, and the first failure returns its NACK immediately. The server first checks the argument format: the player must be `X` or `O`, followed by a comma, then an optional minus sign and digits only, otherwise the reply is `BADARG`. It then checks that the number is within 0–8, otherwise `RANGE`. Next it checks that the player is the current player, otherwise `NOTURN`. Finally it checks that the target cell is empty, otherwise `OCCUPIED`. Because the order is fixed, a request that fails several checks gets only the first error, so `MOVE|X,9` gets `RANGE` even if it is O's turn. The server writes the mark to the board only after all four checks pass, so a rejected move leaves the board, the turn and the state unchanged. After the write, the server checks for a winning line first and replies `WIN|<player>,<board>`. If there is no line and the board is full, it replies `DRAW|<board>`. Otherwise it passes the turn and replies `BOARD|<board>`. Win is checked before draw so that a final move that completes a line counts as a win.

The figure leaves out a few things. Frame-level errors (`BADFRAME`, `BADSEQ`, `BADCRC` and `BADCMD`) are produced before a command reaches the FSM, can occur in any state and never change it. An incomplete frame is discarded after a 500 ms timeout with no response. The reset button SW4 returns the server to `GAME_IDLE` from any state, but the figure draws it only as the power-up transition. The figure also

groups the four move checks under a single “invalid MOVE” label instead of drawing each one.

C. GUI Sends Requests and Receives Responses

The user must first establish a serial connection with the STM32 and once connected, the application provides different controls for the game such as starting a new game, making moves and ending the game. Thus, the user's actions are turned into protocol requests that are sent over to the server, and the server determines whether the request is acceptable.

One of the way in which the GUI can change is with the board refresh. This allows displayed cells to be refreshed and the win/draw response indicates the game has reached a terminal outcome. A NACK indicates that a request was rejected and so the GUI relays the error in the displayed game state too.

D. UART Circuit Diagram

The PC communicates with the STM32 microcontroller through USART1 and the ST-LINK virtual COM port over USB. USART1 uses PA9 as its TX pin (transmit) and PA10 as its RX pin (receive). The serial communication operates at 115200 baud using 8 data bits, no parity and one stop bit (8N1). The board also makes use of SW0 push button connected to PA0, which is ACTIVE-LOW as well as LEDs connect to PB10, PB11, PB0-PB7. SW0 is used to start a new game and the LEDs reflect the game state and board information.

Figure 3 shows the UART connection between the STM32 board and the PC, together with the relevant board connections.

IV. Implementation

A. Receiving UART Bytes Safely

The UART interrupt receives incoming bytes and stores them in a circular buffer for the main loop to process. This keeps the interrupt short and prevents it from spending too much time processing complete messages. The buffer also allows the main loop to process incoming data separately from the interrupt. If the buffer is full, an overflow flag is set instead of overwriting unread data. UART errors are also recorded and cleared so that they can be handled safely outside the interrupt. An obvious alternative would be to process the complete message inside the interrupt, but this could keep the processor in the interrupt for too long and delay other tasks.

```
void uart_rx_isr(void)
{
    uint32_t isr = USART1->ISR;

    if (isr & USART_ISR_RXNE) {
        uint8_t b = (uint8_t)USART1->RDR;
        last_byte_tick = HAL_GetTick();
        uint8_t next = (uint8_t)((rx_head + 1U)
                                 % RX_BUF_SIZE);

        if (next != rx_tail) {
            rx_buf[rx_head] = b;
            rx_head = next;
        } else {
            rx_overflow = 1U;
        }
    }
}
```

Listing 1. Receiving UART bytes using an interrupt and circular buffer (main.c)

B. Handling Incomplete and Oversized Frames

The frame parser assembles incoming bytes into complete messages. Since UART delivers a stream of bytes rather than complete messages, the parser looks for the start marker `@` and uses the newline character to identify the end of a frame. If another start marker arrives before the current frame is complete, the parser restarts from the new marker. It also detects oversized frames and rejects them instead of writing beyond the buffer. An obvious alternative would be to keep appending bytes until a newline arrives, but this could cause a buffer overflow. Incomplete frames are also discarded if no new byte arrives within the timeout period.

```
if (!fp->started) {
    if (b != '@') {
        return PARSING;
    }

    fp->started = true;
    fp->length = 0U;
    fp->outOfBounds = false;
}

if (b == '@') {
    fp->length = 0U;
    fp->outOfBounds = false;
    fp->data[fp->length++] = (char)b;
    return PARSING;
}

if (b == '\n') {
    if (fp->outOfBounds) {
        frameparser_init(fp);
        return FRAME_ERROR;
    }

    fp->data[fp->length] = '\0';
    fp->started = false;
    return FINISHED;
}
```

Listing 2. Assembling and validating incoming frames (frameparser.c)

C. Checking Message Integrity with CRC

The CRC check detects accidental corruption in messages sent between the GUI and the STM32 server. Each byte is processed using the CRC polynomial, allowing the receiver to check whether the received frame matches the expected checksum. This is more reliable than using a simple additive checksum because CRC is designed to detect common patterns of transmission errors. The CRC calculation is implemented explicitly so that the result follows the protocol's defined initial value, polynomial, and processing rules. Using a different CRC implementation without matching these settings could cause valid messages to be rejected.

```
uint16_t crc16_ccitt_false(const uint8_t *data,
                           size_t length)
{
    uint16_t crc = 0xFFFFU;

    for (size_t i = 0; i < length; i++) {
```

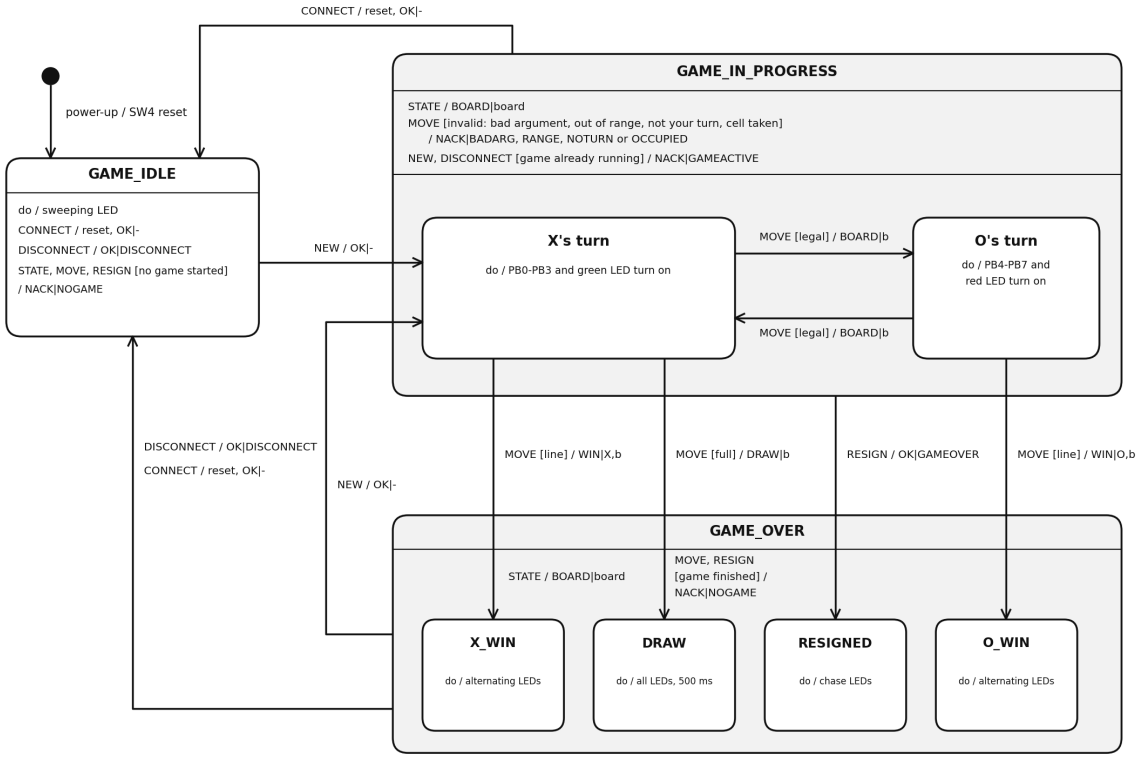


Fig. 2. Server FSM with the move-validation branch.

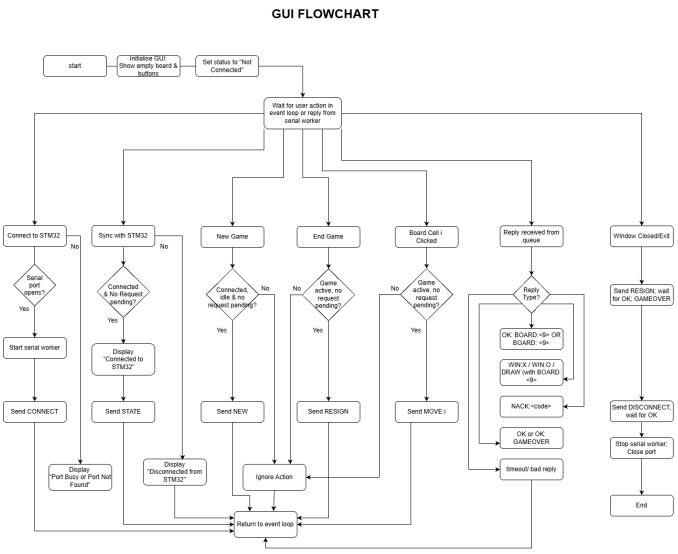


Fig. 3. Flowchart showing the GUI connected to the STM32 during a game of Tic-Tac-Toe.

```
    crc ^= (uint16_t)data[i] << 8;

    for (int bit = 0; bit < 8; bit++) {
        if (crc & 0x8000U) {
            crc = (crc << 1) ^ 0x1021U;
        } else {
            crc <<= 1;
        }
    }
```

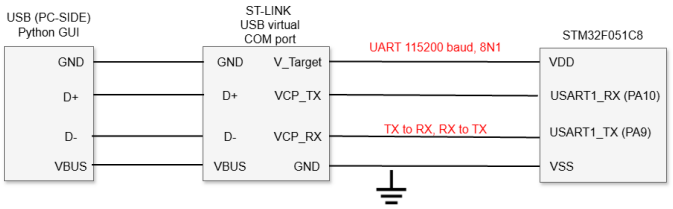


Fig. 4. UART connection between the STM32F051C8 and PC, with the board button and LED connections.

```
        }
    }

    return crc;
}
```

Listing 3. Calculating the CRC-16 checksum (protocol.c)

D. Keeping the GUI Responsive During Communication

The serial communication runs in a separate worker thread so that waiting for a response does not freeze the GUI. Commands are placed in a queue and processed by the worker, which sends them to the STM32 and emits signals when responses or errors are received. This separates communication from the interface and allows the GUI to remain responsive while the serial operation is in progress. An obvious alternative would be to perform serial communication directly in the GUI thread, but waiting for a response could prevent the interface from updating or responding to user input.

```
class SerialWorker(QThread):
    response_received = Signal(str, str)
    error_occurred = Signal(str, str)
    new_connection = Signal(bool)

    def __init__(self, port="COM5"):
        super().__init__()
        self.port = port
        self.commands = Queue()
        self.client = None

    def send_command(self, command, arguments="-"):
        self.commands.put((command, arguments))

    def run(self):
        try:
            self.client = SerialClient(port=self.port)
            self.new_connection.emit(True)

            while True:
                command_data = self.commands.get()
                if command_data is None:
                    break

                command, arguments = command_data
                response = self.client.send_command(
                    command, arguments)
                self.response_received.emit(*response)
        finally:
            if self.client is not None:
                self.client.close()
                self.client = None
            self.new_connection.emit(False)
```

Listing 4. Processing serial commands in a worker thread (serial worker)

E. Matching Replies to the Correct Commands

The GUI keeps track of commands that have been sent using an `inflight` queue. When a response arrives, the GUI removes the oldest command from the queue and uses it to determine how the response should be handled. This is important because different commands can produce responses with the same response type, such as `BOARD`. The GUI therefore needs to know which request the response belongs to instead of relying only on the response type. An obvious alternative would be to handle every response based only on its type, but this could cause the GUI to interpret a response in the wrong context. This approach assumes that replies arrive in the same order as requests; if that assumption changes, responses should instead be matched using their protocol sequence numbers.

```
def send(self, command, arguments="-",
         status="", label=None):
    self.inflight.append(label or command)

    if label != "POLL":
        self.pending = True
        if status:
            self.status_label.setText(status)
        self.update_controls()

    self.worker.send_command(command, arguments)

def on_response(self, worker, command, arguments):
    if worker is not self.worker or self.lost:
        return

    sent = self.inflight.popleft() if self.inflight else ""

    if command == "NACK":
        self.handle_nack(sent, arguments)
    elif sent == "MOVE":
        self.handle_move_reply(command, arguments)
    elif sent in ("STATE", "SYNC", "POLL") and command == "
     BOARD":
        self.handle_board_reply(sent, arguments)
```

Listing 5. Associating responses with their requests (GUI client)

V. Game Logic and Server Authority

A. Move Validation Order

Every frame that reaches the board is checked before the server looks at what it asks for. The server confirms that the frame has the right shape, that the sequence number is valid hexadecimal, and that the CRC matches. If any of these fail, it replies with a `NACK` and goes no further. An unknown command is also answered with `NACK|BADCMD`. Only a frame that passes all of this reaches the `MOVE` handler in `handle_frame()` in `main.c`.

The `MOVE` handler then runs five checks in a fixed order, and the first one that fails sends its error and returns. The server first checks that a game is running. If not, it answers `NOGAME`. It then checks the argument, which must be a mark (`X` or `O`), a comma and a number made only of digits. Anything else gets `BADARG`. The number is then checked to be between 0 and 8, and a valid number outside that range gets `RANGE`. After that the server compares the mark in the request with `current_player`, and a mismatch gets `NOTURN`. Last, it checks that the chosen cell still holds a `.`, and a taken cell gets `OCCUPIED`.

Only after all five checks pass does the server write the mark to `game_board`. The order is deliberate. The index is checked for range before it is used, so a bad number can never be used to address the board array. The turn is checked before the cell, so an out-of-turn move onto a taken square is reported as `NOTURN`. Because the server does all the checking, a client that sends bad moves is simply answered with an error and the board stays as it was. //

The server checks whether `game_state = GAME_IN_PROGRESS`. Following that, it checks that the argument format is valid `e.g` X, i.
The next thing that is check is that the digit i is correctly converted into a cell index, rejecting any values that are greater than 8 with a `NACK|RANGE`. The server then compares the potential next player with the current player and ensures there is no mismatch. Lastly, the server checks if the board cell chosen contains a `.`, indicating an empty cell. It is only once all these checks have passed that the server writes/displays the move to the game board. The ordering of these checks is vital in ensuring that, for example, an invalid index is rejected before it is used to access the game board cells.

B. Why a Rejected Request Cannot Change the Game State

The game state is four variables: `game_board`, `current_player`, `game_state` and `game_result`. In `main.c` they are written in only four places: `start_new_game()`, `reset_to_idle()`, the `MOVE` branch after its checks, and the `RESIGN` branch. The argument is as follows.

1) **Framing errors.** FRAME_ERROR, BADFRAME, BADSEQ and BADCRC are produced in the main loop or at the top of handle_frame(), before any command handler runs. That code touches only the parser and the receive buffer.
2) **UART errors.** The ISR only sets flags. The main loop responds by clearing the receive buffer and calling frameparser_init().
3) **Unknown commands.** The final else contains a single call to send_frame() with BADCMD.
4) **MOVE.** The number is parsed into the locals value, negative and move_index. The first assignment to game_board comes after the NOGAME, BADARG, RANGE, NOTURN and OCCUPIED checks, and each of those ends in return.
5) **NEW.** The GAMEACTIVE check returns before start_new_game() is called.
6) **RESIGN and DISCONNECT.** The NOGAME and GAMEACTIVE checks return before the state assignment or reset_to_idle().
7) **STATE.** It only reads the board. CONNECT has no rejection path.

Every NACK is followed directly by return, and in every handler the first write comes after the last check. So a path that ends in an error cannot have written to the game state. handle_frame() does edit the frame buffer by placing \0 at the separators, but that buffer belongs to the parser and is reset after every frame.

These tests check the argument. Each one rejects a request and then confirms that the state is unchanged.

- test_moves.py sends an occupied cell, an out-of-turn move and cell 9. It then reads STATE and finds the board unchanged.
- test_invalid_commands.py sends MOVE before NEW, a non-numeric cell, a two-digit cell and an unknown command. After each one the board is read and is still unchanged.
- test_bad_crc.py sends a corrupted MOVE, then confirms the board and the turn are preserved.
- test_disconnect_states.py rejects DISCONNECT during a game and shows that the game continues.
- led_demo.py confirms on the hardware that rejected moves do not change the LCD or LEDs.

C. Turn Control

The server alone tracks the turn in the variable current_player, which is 'X' at the start of each game. Every MOVE carries the mark of the player making it, and the server compares that mark with current_player. A mismatch returns NACK|NOTURN. The turn changes only at the end of a legal move that did not finish the game, so a rejected move cannot pass the turn. The turn check comes before the occupied-cell check, so an out-of-turn move onto a taken cell is answered with NOTURN. This is tested in test_moves.py, test 5, where O tries to move twice in a row.

D. Win and Draw Detection

The board is stored as nine cells numbered 0 to 8, left to right and top to bottom. A player wins by holding three cells in a line, and there are eight such lines. They are the three rows (0-1-2, 3-4-5 and 6-7-8), the three columns (0-3-6, 1-4-7 and 2-5-8) and the two diagonals (0-4-8 and 2-4-6). After every legal move, check_winner() tests all eight lines, but only for the player who has just moved, because that is the only player whose marks have changed.

The server handles a legal move in a fixed order. It writes the mark to the board, checks whether the mover has won, checks whether the board is full, and only then switches the turn. The win check has to come before the draw check, because the ninth move can fill the board and complete a line at the same time. If the full-board check ran first, a game won on the last square would be reported as a draw.

When the game ends in a win or a draw, game_state is set to GAME_OVER. From that point any MOVE fails the first validation check and is answered with NACK|NOGAME, so nothing can be added to a finished board. It stays that way until the client sends NEW. test_all_wins.py plays all eight lines and checks after each win that a further move is refused. test_win_draw.py checks a win and a draw, and that moves are rejected after each of them.

E. Why the Client Holds No Game Rules

The GUI sends the square the player clicked and displays whatever the server answers. It does not decide whether a move is legal, whether someone has won or whether the game is over. The only checks in main.py are conveniences. It does not send a click on a square it already shows as filled, and it works out the mark from the piece counts. Neither is relied on, because the server repeats every check. After a rejected move the client sends STATE and redraws from the server's board. The winning line is highlighted only after the server has sent WIN.

If the client held the rules, there would be two copies of them that could drift apart. A modified or second client could disagree with the board, and a stale screen could accept moves that the server would refuse. With the rules only on the board, the server's state is the single source of truth.

VI. Protocol Robustness and Integrity

The server checks frames and commands coming in, rejects invalid input with the appropriate response messages and manages the parser for incomplete frames or buffer overflows.

A. Partial Frames and Timeouts

The timeout rule we chose to use was a value of 500 ms. The server completely discards the incomplete frame after 500 ms of inactivity. This is measured using HAL_GetTick(), and if the parser does not receive any further bytes within the timeout

TABLE V
BAD INPUT, THE SERVER'S HANDLING, AND THE TEST THAT CHECKS IT.

Bad input	Handling and response	Test
Partial frame	Discarded after 500 ms of silence; a new `@` also restarts frame collection	`test_frames.py` (tests 9–10)
Oversized frame	Extra bytes are discarded until newline; parser returns `FRAME_ERROR` and server sends `NACK|BADFRAME`	`test_frames.py` (test 8)
Garbage bytes	Bytes are ignored until the `@` start marker	`test_frames.py` (test 7)
Wrong case or unknown command	Unmatched commands are rejected with `NACK|BADCMD`	`test_invalid_commands.py`
Corrupted bytes (checksum)	CRC validation rejects the frame with `NACK|BADCRC`	`test_bad_crc.py`
UART errors (overrun, framing)	Receive-error flags trigger buffer clearing and parser reset	No dedicated automated test

period, `frameparser_init(&fp)` resets the parser for a new frame to be received. This timeout rule helps prevent an incomplete frame from getting stuck in the parser for an indefinite period of time.

VII. VALIDATION

A. Automated test harness on the real board

Every test is a short Python script that talks to the STM32F051C8 over its USB serial port using pySerial [1]. No human input is needed while a script runs. The scripts are plain Python with `assert` statements, not a pytest suite. A failed check raises an `AssertionError`, prints the failing step and stops the script. A passing script ends with a "...PASSED" line. All results below come from the real board: no simulator was used.

Connection. Each script opens `COM5` at 115200 baud (8N1) with a 1–2 s read timeout. It sends one frame, waits for exactly one reply line with `readline()`, then asserts on the reply. An empty reply (timeout) counts as a failure ("No response"), so a silent board cannot pass.

Frames. The scripts build frames in the same format as the PC GUI, `@SSSS|COMMAND|ARGUMENTS|CCCC` followed by a newline, where `SSSS` is a four-digit hexadecimal sequence number and `CCCC` is the CRC-16/CCITT-FALSE of the text between `@` and the last `|`. The CRC is computed with Python's `binascii.crc_hqx` (initial value `0xFFFF`). This is an independent implementation from the C function `protocol_crc16` on the board, so a matching reply also shows that the two CRCs agree. The LED demo script uses its own hand-written CRC routine for the same reason.

Known starting state. The first thing every script does is send `CONNECT` with sequence number `0x0FFF`. `CONNECT` always returns the server to IDLE, so a test never depends on what the previous test left behind. The scripts can run in any order, with no manual reset of the board.

Running. Close any other program that holds the port (the GUI, a serial terminal), then run one script, for example `python test_moves.py`. All ten scripts were run in one PowerShell loop with the output saved to text files.

Visual check. The LEDs and LCD cannot be read over the serial port. A separate script, `led_demo.py`, sends the commands for each game state and asserts the replies, then pauses so the tester can confirm the LED pattern by eye. This part is a manual check, not an automated one.

B. All required cases pass on the board

Table VI maps each required test from the brief to a named script and test number. All required tests pass. Each negative test also checks that the board did not change. For example, after an occupied-cell move, a wrong-turn move and an out-of-range move, `STATE` still returns `XO.......`. A rejection is only correct if it has no side effect.

Two replies are documented equivalents of the brief's wording. `NEW` returns `OK|-` rather than a board, so "NEW returns an empty board" is tested as `NEW` followed by `STATE`, which returns `..........`. A `STATE` request before any game has started returns `NACK:NOGAME`, the same as a move before `NEW`.

The scripts go beyond the required cases:

- `all_wins` tests all eight winning lines (three rows, three columns, two diagonals). Each win returns `WIN|X,<board>`, and a further move is then refused with `NACK:NOGAME`.
- `win_draw` fills the board with no winning line and receives `DRAW|XXOOOXXOX`.
- `bad_crc` corrupts one CRC digit. The board answers `NACK:BADCRC`, keeps its board, and accepts the next valid command.
- `frames` checks the framing layer: garbage before `@` is ignored, an 80-byte frame returns `NACK:BADFRAME`, an incomplete frame is dropped after the timeout without a reply, and a frame sent one byte every 2 ms is still received intact.
- `invalid_commands` checks `NACK:BADCMD` for an unknown command and `NACK:BADARG` for a non-numeric cell. It also checks that `NEW` is refused during a game (`NACK:GAMEACTIVE`) and accepted after `RESIGN`.
- `state_rules`, `disconnect_states` and `resign_disconnect` check which commands are allowed in IDLE, during a game and after game over. For example, `DISCONNECT` is refused during a game and the game then continues.

All ten scripts finished with their "PASSED" line on the final run (Figures 5 shows one of the tests).

C. What testing revealed

- **Dropped bytes.** Receiving with `HAL_UART_Receive_IT` dropped bytes, so frames were lost. A small script, `diag.py`, sent the same `CONNECT` frames two ways: whole, and one byte every 2 ms. We replaced the HAL call with a register-level USART1 interrupt handler and a ring buffer. The one-byte-at-a-time case is now a permanent test (`frames`, Test 10).

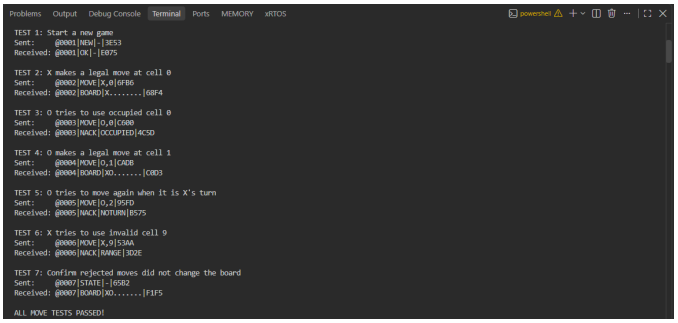


Fig. 5. Output of the move tests on the board: legal moves, occupied cell, wrong turn, out-of-range cell, and an unchanged board.

TABLE VI
THE BRIEF'S REQUIRED TESTS AND THEIR RESULTS ON THE BOARD.

Requirement (brief, p. 4)	Test (script, step)	Result
CONNECT returns OK; NEW returns an empty board	`serial_client` T1–T3	pass
A legal move changes the board exactly as expected	`moves` T2, T4	pass
Occupied cell returns NACK:OCCUPIED, board unchanged	`moves` T3, T7	pass
Out-of-range move returns NACK:RANGE	`moves` T6; `invalid_commands` T3	pass
Move after game over returns NACK:NOGAME	`win_draw` T2, T4; `all_wins`	pass
Scripted win and draw give the correct result	`win_draw` T1, T3; `all_wins`	pass
AI never makes an illegal move (if implemented)	—	n/a

- **Tests depended on each other.** In the first full run, the script that tested the serial client left a game in progress. Every later script then started inside that game, and its `NEW` was refused with `NACK:GAMEACTIVE`. Most scripts failed, and one passed only because the leftover game happened to be empty. We fixed this by starting every script with `CONNECT`, which also removed the need to reset the board by hand.
- **Tests lagged the specification.** We later decided that `NEW` is accepted only in IDLE and after game over, so the old "NEW resets the board" test was wrong. It now asserts `NACK:GAMEACTIVE`, then ends the game with `RESIGN` and starts a new one. Likewise, `STATE` with no game returns `NACK:NOGAME`, and the state-rule test was updated to match.
- **Checks that only printed.** Several early checks (`STATE` before `NEW`, the cell `A`, the cell `10`) printed the reply but did not assert it. They now assert `NACK:NOGAME`, `NACK:BADARG` and `NACK:RANGE` respectively.

VIII. Conclusion

The project successfully implemented a Tic Tac Toe game in which the STM32 board controls the game rules and the GUI allows the player to interact with it. The server checks each move, keeps track of whose turn it is, detects wins and draws, and rejects invalid moves. Testing on the physical board showed that the game handled normal play and situations such as occupied squares, incorrect commands, invalid messages and attempts to play after the game had ended. The LCD and LEDs also provided feedback about the game.



Fig. 6. Screenshot showing the GUI connected to the STM32 board.

Usefulness. This design could be useful in other systems where a computer communicates with a microcontroller. For example, a computer could be used to control a robot or a machine, while the microcontroller checks that each instruction is allowed before carrying it out. Our project demonstrates why this separation is useful: the GUI handles what the player sees and does, while the STM32 board makes the decisions. This means that the game rules are not dependent on the GUI behaving correctly.

Limitations. The main limitation is that the system currently uses one serial connection to communicate with one GUI. This means we cannot open separate game windows on two different devices and have two players play against each other through those windows. Both players must use the same GUI. The game also only supports Noughts(O) and Crosses(X), has no computer-controlled opponent, and does not save players' results between games. In addition, communication depends on a working serial connection, so a disconnected cable or a communication failure can interrupt a game. Although the tests covered many expected situations, more testing would be needed to understand how the system behaves during repeated disconnections or other unexpected hardware problems.

Improvements. There are several ways the project could be improved. An AI opponent would allow a player to play without needing another person. A leaderboard could record

wins, losses and draws, giving players a reason to return and improve their results. The GUI could also be extended to support two separate windows on different devices, with a suitable communication method allowing both players to connect to the same game server. These additions would make the game more enjoyable to use while building on the system developed in this project.

IX. Video

The demonstration video is available at: *"link to video demonstration"*.

A. The video shows the game starting in the idle state, followed by a complete game ending in a win, with further moves blocked after game over, and a second game ending in a draw. A new game is then started to demonstrate the error message when the board is disconnected, the game continuing after reconnection, and the application closing using the Exit button.

X. Code

The source code is at: *"link to source code"*. The repository contains a README.md file in the repository for further build and run instructions.

References

[1] C. Liechti, “pySerial documentation,” https://pyserial.readthedocs.io/.